what is machine learning ? and its work flow?

Machine Learning

Machine Learning (ML) is a branch of AI where computers learn patterns from data and make predictions or decisions without being explicitly programmed.

Instead of writing all rules manually, the machine learns automatically from examples and data

Simple Definition

Machine Learning = Teaching computers to learn from data.

Real-Life Examples

YouTube recommendations

Spam email detection

Face recognition

Netflix movie suggestion

Types of Machine Learning

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

#1. Supervised Learning

The model learns using labeled data.

Labeled data means:

Input data + Correct output already given

Examples

Student marks prediction

House price prediction

Spam email detection

Algorithms

Linear Regression

Logistic Regression

Decision Tree

Simple Idea

Learning with a teacher.

2. Unsupervised Learning

The model learns using unlabeled data.

The machine finds hidden patterns by itself.

Examples

Customer segmentation

Product grouping

Algorithms

K-Means Clustering

PCA

Simple Idea

Learning without a teacher.

3. Reinforcement Learning

The model learns through rewards and punishments.

The system improves by trial and error.

Examples

Self-driving cars

Game-playing AI

Robots

Simple Idea

Learning from experience.

define startegy

A strategy is a plan or method used to achieve a goal successfully.

It helps in deciding:

What to do

How to do

When to do

Simple Definition

Strategy = A smart plan to achieve an objective.

data collection

Data collection is the process of gathering information or data from different sources for analysis and decision-making.

In Machine Learning, data collection is the first and most important step.

Simple Definition

Data Collection = Gathering useful data from various sources.

Machine Learning Workflow

Step-by-Step Workflow

Collect Data

```
    ↓
```

Clean & Prepare Data

```
    ↓
```

Choose ML Model

```
    ↓
```

Train Model

```
   ↓
```

Test/Evaluate Model

```
↓
```

Deploy Model

```
↓
```

Prediction/Output

define strategy

```
 ↓
```

data collection

```
  ↓
```

data modeling

```
  ↓
```

evaluation

```
  ↓
```

optimization

```
   ↓
```

deployment

```
   ↓
```

performance monitering

sklearn workflow

Step 1:- Collect and Understand Data

The hospital collected the patient's CT and MRI scan reports to understand the finger injury.

Step 2:- Clean and Prepare the Data

The scan images were improved and prepared so the nerve area could be seen clearly.

Step 3:- Train a Model

The AI model was trained using many old medical scan images of nerve injuries.

Step 4:- Evaluate if it is Accurate

The AI checked the scans and identified the nerve cut that normal checking methods missed.

Step 5:- Deploy and Use it in Real World

The hospital used the AI system to help doctors find nerve injuries and give better treatment.

step 1: Import

step 2: prepare data

step 3: train the model

step 4:

Double-click (or enter) to edit

```
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score , classification_report
import pandas as pd
import numpy as np
```

```
# from sklearn.datasets import load_iris , load_digits
```

```
# data={
#     'hours_studied': [1,2,3,4,5,6,7,8,9,10],
#     'pass_fail': ['no','yes','yes','yes','yes','yes','yes','yes','yes','yes']
# }
# df=pd.DataFrame(data)

# #separate
# x = df[['hours_studied']]
# y = df['pass_fail']

# # #split()
# x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
# print(f'Train: {len(x_train)} rows | Test: {len(x_test)} rows')
# # #create and train
# model=LogisticRegression()
# model.fit(x_train,y_train)
# print(" model trained successfully")

# # evaluate on test set
# y_pred = model.predict(x_test)
# accuracy = accuracy_score(y_test, y_pred)
# print(f'Accuracy: {accuracy:.2%}')

# print(classification_report(y_test,y_pred,target_names=['fail', 'pass'], labels=['no', 'yes']))

data={
  "hours_studies":[1,2,3,4,5,6,7,8,9,10],
  "Pass_fail":[0,0,0,1,1,1,1,1,1,1]
}
df=pd.DataFrame(data)
print(df)
#seperate
x=df[['hours_studies']]
y=df['Pass_fail']

#split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print(f"train :{len(x_train)} |test :{len(y_test)}")
#train the model
model=LogisticRegression()
model.fit(x_train,y_train)
print("model trained sucessfully")
#prediction
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print(f"accuracy:{accuracy:.2%}")
print(classification_report(y_test,y_pred,target_names=["pass",'fail']))
```

```
   hours_studies  Pass_fail
0              1          0
1              2          0
2              3          0
3              4          1
4              5          1
5              6          1
6              7          1
7              8          1
8              9          1
9             10          1
train :8 |test :2
model trained sucessfully
accuracy:100.00%
```

```
              precision    recall  f1-score   support

        pass       1.00      1.00      1.00         1
        fail       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

```
new_students=[[2],[1],[5],[8]]
predictions=model.predict(new_students)
labels={0:'FAIL',1:"PASS"}
for hrs,pred in zip(new_students,predictions):
  print(f"Studied { hrs [0] } hrs-> {labels[pred]}")
```

```
Studied 2 hrs-> FAIL
Studied 1 hrs-> FAIL
Studied 5 hrs-> PASS
Studied 8 hrs-> PASS
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
  warnings.warn(
```

```
data = {
    "hours_studies": [1,2,3,4,5,6,7,8,9,10],
    "Pass_fail": [1,0,1,0,1,0,1,0,1,0]
}

# Create DataFrame
df = pd.DataFrame(data)
print(df)

# Separate input and output
x = df[['hours_studies']]
y = df['Pass_fail']

# Split data
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

print(f"\nTrain : {len(x_train)} | Test : {len(x_test)}")

# Train model
model = LogisticRegression()
model.fit(x_train, y_train)

print("\nModel trained successfully")

# Prediction on test data
y_pred = model.predict(x_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy : {accuracy:.2%}")

# Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["FAIL", "PASS"]))
```

```
   hours_studies  Pass_fail
0              1          1
1              2          0
2              3          1
3              4          0
4              5          1
5              6          0
6              7          1
7              8          0
8              9          1
9             10          0

Train : 8 | Test : 2

Model trained successfully

Accuracy : 0.00%

Classification Report:
              precision    recall  f1-score   support

        FAIL       0.00      0.00      0.00       1.0
        PASS       0.00      0.00      0.00       1.0
```

```
    accuracy                           0.00       2.0
   macro avg       0.00      0.00      0.00       2.0
weighted avg       0.00      0.00      0.00       2.0
```

```
new_students = [[2], [1], [5], [8]]

predictions = model.predict(new_students)

labels = {
    0: "FAIL",
    1: "PASS"
}

print("\nNew Student Predictions:")
for hrs, pred in zip(new_students, predictions):
    print(f"Studied {hrs[0]} hrs -> {labels[pred]}")
```

```
New Student Predictions:
Studied 2 hrs -> PASS
Studied 1 hrs -> PASS
Studied 5 hrs -> PASS
Studied 8 hrs -> FAIL
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
  warnings.warn(
```